

Owned-Submodule Build + Test

Health Audit — 2026-07-09

Revision: 1 **Last modified:** 2026-07-09T00:00:00Z **Scope:** submodules/owned Go + Android bricks (§11.4.28 equal-codebase). submodules/helixqa EXCLUDED (pre-existing uncommitted changes — untouched per instruction). **Authority:** Helix Constitution §11.4.28 (owned submodules are equal codebase, must build + test green from their own module roots), project CLAUDE.md (“go build ./..., gofmt, go vet MUST be clean”).

Method (captured evidence)

For every Go module root, from that root: gofmt -l ., go build ./..., go vet ./..., go test ./... -count=1. Go toolchain: go1.26.4. Android/Gradle bricks were statically inspected only (no gradlew wrapper present + a full AGP/Android build is heavy and needs the Android SDK — honest boundary per §11.4.6). Results below are the literal captured PASS/FAIL of those commands.

Per-brick health table

Brick	Type / module root (marker)	build 	vet 	gofmt clean 	test 
ota-protocol	Go — ota-protocol/go.mod (github.com/HelixDevelopment/ota-protocol)			1 file 	
ota-artifact-validator	Go — ota-artifact-validator/go.mod				

Brick	Type / module root (marker)	build	vet	gofmt clean	test
ota-rollout-engine	Go — ota-rollout-engine/ go.mod				
ota-telemetry-schema	Go — ota-telemetry-schema/ go.mod				
challenges	Go — challenges/go.mod (digital.vasic.challenges)			pre-existing	(FIXED)
containers	Go — containers/go.mod (digital.vasic.containers)			34 files	
doc_processor	Go — doc_processor/go.mod (digital.vasic.docprocessor)				
http3	Go — http3/go.mod (digital.vasic.http3)			1 file	
llm_orchestrator	Go — llm_orchestrator/go.mod (digital.vasic.llmorchestrator)			6 files	
llm_provider (LLMProvider = symlink)	Go — llm_provider/go.mod (digital.vasic.llmprovider)				
llms_verifier	Go — llms_verifier/go.mod (llmsverifier)			211 files	
llms_verifier/ llm-verifier	Go — nested llms_verifier/llm- verifier/go.mod (digital.vasic.llmsverifier)			177 files	RED

Brick	Type / module root (marker)	build	vet	gofmt clean	test
		✓	✓	▲	✓
security	Go — security/go.mod (digital.vasic.security)			13 files	
vision_engine	Go — vision_engine/go.mod (digital.vasic.visionengine)			17 files	
ota-android-agent	Android/Kotlin — build.gradle.kts + settings.gradle.kts (AGP + kotlin.android)	—	—	—	—
ota-update-engine-bridge	Android/Kotlin — build.gradle.kts + settings.gradle.kts	—	—	—	—

Legend: pass · fail · non-fatal drift/note · — not applicable/not run.

Fix applied — challenges brick (RED → GREEN)

Chosen brick: challenges — it was the highest-value FIXABLE gap: the only brick whose `go test ./...` exited non-zero. Two genuine defects were root-caused (§11.4.102) and fixed; scope kept to this one brick.

Defect 1 — tests/chaos/chaos_test.go

TestChaosConfigEdgeCases/extreme-long-path (deterministic FAIL)

- **Root cause:** the subtest `built longPath := filepath.Join(t.TempDir(), string(make([]byte, 500)))`. `string(make([]byte, 500))` is a **single path component of 500 NUL bytes**. NUL is illegal in every Unix path component (→ EINVAL) and 500 also exceeds NAME_MAX (255). `BaseChallenge.Configure` →

`os.MkdirAll(ResultsDir)` can therefore **never** succeed on any Linux filesystem, yet the test asserted success. This is a §11.4.1 FAIL-bluff: the test failed for a **test-bug** reason, not a product defect.

- Before (captured): `chaos_test.go:151: extreme-long-path: Configure failed: create results dir .../<500 NUL bytes>: mkdir ...: invalid argument → FAIL`
`digital.vasic.challenges/tests/chaos.`
- **Fix:** replaced the impossible input with a **genuinely long BUT filesystem-valid** path — 3 nested components of 200 as each (each < NAME_MAX, total < PATH_MAX) so `Configure` truly exercises long-path handling and succeeds. Added a second subtest `invalid-path-degrades-gracefully` feeding a NUL-containing path and asserting `Configure` returns an **error** (never panics) — preserving the chaos-suite’s own contract (“must always degrade gracefully”) and the “extreme” intent (§11.4.27 real coverage, not a fake).
 - After (captured): `--- PASS: TestChaosConfigEdgeCases/extreme-long-path + --- PASS: .../invalid-path-degrades-gracefully + --- PASS: .../unconfigured-validate → ok`
`digital.vasic.challenges/tests/chaos.`

Defect 2 — `pkg/challenge TestChaosConcurrentResultWrite` (intermittent FAIL — genuine product race)

- **Discovery:** surfaced under discovery-pressure (§11.4.118) on the full-brick run; the first probe run did NOT show it → nondeterministic. Not labelled “flaky” — root-caused per §11.4.102.
- **Root cause:** exported `(*Result).RecordAction(pkg/challenge/antibluff.go)` did an **unsynchronized** `r.RecordedActions = append(...)`. `TestChaosConcurrentResultWrite` spawns 30 goroutines calling it concurrently; the un-locked append loses updates (a data race), so `len(RecordedActions) < 30` intermittently. This is a real thread-safety defect in a method consumed downstream (HelixQA / LLMsVerifier), violating dev-principle #3 “always consider concurrent callers”; the action-trace is exactly what the anti-bluff validator counts, so a lost action can silently turn a real PASS into a bluff.
 - Before (captured): `chaos_test.go:384: expected at least 30 recorded actions, got 28 → FAIL` `digital.vasic.challenges/pkg/challenge.`
- **Fix:** added an unexported `mu sync.Mutex` to `Result` (`pkg/challenge/result.go`) and wrapped the append in `r.mu.Lock()/Unlock()`. Unexported → `encoding/json` ignores it, so serialized `Results` are byte-unchanged (verified by the `report/base` round-trip tests still passing). No copylocks risk: audited the module — `Result` is used by

pointer everywhere (return &result), never returned/copied by value, so go vet stays clean.

- After (captured): go vet ./pkg/challenge/ rc=0; go test ./pkg/challenge/ -run 'TestChaosConcurrentResultWrite|TestRecordAction' -race -count=5 → all PASS (deterministic + race-clean); full brick go test ./... -count=1 rc=0.

Files changed (challenges brick only)

- tests/chaos/chaos_test.go — fixed subtest + new graceful-degradation subtest + strings import.
- pkg/challenge/antibluff.go — lock the RecordAction append (+ gofmt: this file was pre-existing gofmt-unclean and is now clean; gofmt also normalized • → - doc-list bullets per Go 1.19+).
- pkg/challenge/result.go — add unexported mu sync.Mutex field + sync import.

Verification: gofmt -l clean on all three edited files; go build ./... rc=0; go vet ./pkg/challenge/ rc=0; full-brick go test ./... -count=1 rc=0.

Discovered, not fixed (honest boundary — §11.4.6 / §11.4.118)

- **llms_verifier/llm-verifier is RED** — TestCommandFlagValidation (models list, providers list with flags) and TestOutputFormats (models/providers/results list, json+table) FAIL. Different brick; not in the single-brick fix scope this pass. Recommend a dedicated pass: root-cause whether the CLI output/flag contract changed vs the test's expectation (build+vet are green, so it is a behavioral/CLI-contract mismatch, not a compile break).
- **ota-protocol gofmt drift** — payload_fuzz_test.go is gofmt-unclean (comment alignment) in the core OTA brick; a 1-command gofmt -w fix, left untouched to preserve single-brick scope. Worth a quick follow-up given the project mandate “gofmt MUST be clean”.
- **Pre-existing gofmt drift** across containers (34), security (13), vision_engine (17), llm_orchestrator (6), http3 (1), and the two llms_verifier modules (211 / 177). All build/vet/test green; formatting-only, out of scope for this pass.
- **Android bricks** (ota-android-agent, ota-update-engine-bridge) verified structurally only; a real Gradle/AGP build was not run (heavy, needs Android SDK, no gradlew present).

Sources verified

Verification method for the fix was **local reproduction + captured tool output** (§11.4.123 rock-solid proof), which is stronger than doc citation for these claims; the standard references relied on: - POSIX / Linux path limits — NAME_MAX = 255 bytes per component, PATH_MAX = 4096; NUL (0x00) is not permitted in a pathname component (open(2)/path_resolution(7) → EINVAL). Confirmed empirically by the captured `mkdir ...: invalid argument failure`. - Go sync package — `sync.Mutex` zero-value is ready-to-use; unexported struct fields are omitted by `encoding/json`. Confirmed by the unchanged JSON round-trip tests passing. - Go `gofmt` (Go 1.19+) — normalizes doc-comment list bullets (`•` → `-`); observed in the captured `gofmt -d diff`. - Helix Constitution §11.4.1 (FAIL-bluffs forbidden), §11.4.28 (owned-submodule equal codebase), §11.4.102 (root-cause before fix), §11.4.118 (discovery-pressure), §11.4.6 (no guessing / honest boundary).